

Big Data Cassandra

Laurent d'Orazio

2025-2026

Outline

- Context
- Data Model
- Language

Outline

- Context
- Data Model
- Language

Context

- Initiated by Facebook, now an Apache project distributed by Datastax
 - System first based on Google BigTable (column store)
 - Evolution as an extended relational model (N1NF = Non First Normal Form)
 - Strong types (one of the rare NoSQL systems of this kind)
 - CQL (Cassandra Query Language) to define schema and queries (**no join**)
- Scalable and distributed system
 - Consistent hashing
 - Fault tolerance via replication on multi-nodes mode.

Outline

- Context
- Data Model
- Language

Model

- Keyspaces with tables (a.k.a. column families)
- Table
 - Set of rows consisting of simple or complex values
- Perspectives
 - Perspective A: extended relational realizing the first norm rule (atomic type)
 - Perspective B: row as a structured document
- Warning
 - Confusing terminology: columns, column families, etc.

Pairs and documents

- Pair (key, value)
 - Basic structure
 - Key: an identifier
 - Value: atomic (integer, string) or complex (dictionary, set, list)
- Document (row)
 - Identifier (row key) associated to a set of pairs (key, value)
- Notes
 - Rows are typed with a schema, even nested data
 - Inserting data which do not respect the schema is forbidden (at least with CQL)

Tables and database

- Table (formerly column family)
 - Set of rows conform to the schema of the table
- Keyspace
 - Set of tables
- Notes
 - Columns and super columns seem to disappear

Schema

- Practices
 - No join in Cassandra
 - Data should be grouped in rows (autonomous document)
 - Conception should be based on the most frequent queries of the application
- Note
 - Worst case different forms for the same data (redundancy)
 - Example
 - Movies with directors and actors
 - Artists with movies

Application

- Keyspace creation

- `CREATE KEYSPACE IF NOT EXISTS Movies`
- `WITH REPLICATION = {'class' : 'SimpleStrategy', 'replication_factor': 3};`

- Table creation (without nested data)

```
create table artists (  
    id text,  
    last_name text, first_name text,  
    birth_date int, primary key (id)  
);
```

Insert (1)

- SQL like approach

```
insert into artists (id, last_name, first_name, birth_date)
values ('artist1', 'Belmondo', 'Jean-Paul', 1933);
insert into artists (id, last_name, first_name, birth_date)
values ('artist2', 'Baye', 'Nathalie', 1948);
insert into artists (id, last_name, first_name)
values ('artist3', 'Marceau', 'Sophie');
```

Insert (2)

- JSON

```
insert into artists JSON '{  
  "id": "artist4",  
  "last_name": "Coppola",  
  "first_name": "Sofia",  
  "birth_date": "1971"  
}'
```

With nested data

Complex type

- Type creation

```
create type artist (  
    id text,  
    last_name text,  
    first_name text,  
    birth_date int,  
    role text  
);
```

With nested data

Table creation

- Table creation (with nested data)

```
create table movies (  
    id text,  
    title text,  
    year int,  
    genre text,  
    country text,  
    director frozen<artist>,  
    actors set< frozen<artist>>,  
    primary key (id)  
);
```

Insert documents

- From a JSON document

```
INSERT INTO movies JSON '{
  "id": "movie:1",
  "title": "Vertigo",
  "year": 1958,
  "genre": "drama",
  "country": "USA",
  "director": {
    "id": "artist:3",
    "last_name": "Hitchcock",
    "first_name": "Alfred",
    "birth_date": "1899"
  }
}';
```

Outline

- Context
- Data Model
- Language

Select

```
SELECT  
FROM table_name  
WHERE  
GROUP BY  
ORDER BY  
PER PARTITION LIMIT  
LIMIT  
ALLOW FILTERING
```

Outline

- Context
- Data Model
- Language
- Data Management

Data Management

Table (1)

- Cost to find “leo”?
 - 1
- Cost to find “tom”?
 - 2
- Cost to find “amy”?
 - 6

KEY
“leo”
“tom”
“dan”
“mia”
“zoe”
“amy”

Data Management

Table (2)

- Table of size N
- Complexity
 - $O(N)$

KEY
"leo"
"tom"
"dan"
"mia"
"zoe"
"amy"

Data Management

Hash table/map (1)

- Hash table/map
 - Sum as the position

KEY	HASH
"leo"	
"tom"	
"dan"	
"mia"	
"zoe"	
"amy"	

Example:

laurent = 12+1+22+18+5+14+20 = 92

A	1
B	2
C	3
D	4
E	5
F	6
G	7
H	8
I	9
J	10
K	11
L	12
M	13
N	14
O	15
P	16
Q	17
R	18
S	19
T	20
U	21
V	22
W	23
X	24
Y	25
Z	26

Data Management

Hash table/map (2)

- Hash table/map
 - Sum as the position

KEY	HASH
"leo"	32
"tom"	48
"dan"	19
"mia"	23
"zoe"	46
"amy"	39

Data Management

Hash table/map (3)

- Example
 - 6 buckets

KEY	HASH	HASH mod 6
"leo"	32	
"tom"	48	
"dan"	19	
"mia"	23	
"zoe"	46	
"amy"	39	

Data Management

Hash table/map (4)

- Example
 - 6 buckets

KEY	HASH	HASH mod 6
"leo"	32	2
"tom"	48	0
"dan"	19	1
"mia"	23	5
"zoe"	46	4
"amy"	39	3

BUCKET 0	BUCKET 1	BUCKET 2	BUCKET 3	BUCKET 4	BUCKET 5
"tom"	"dan"	"leo"	"amy"	"zoe"	"mia"

Data Management

Hash table/map (5)

- Cost to find “leo”?
 - 1
- Cost to find “tom”?
 - 1
- Cost to find “amy”?
 - 1

KEY	HASH	HASH mod 6
“leo”	32	2
“tom”	48	0
“dan”	19	1
“mia”	23	5
“zoe”	46	4
“amy”	39	3

BUCKET 0	BUCKET 1	BUCKET 2	BUCKET 3	BUCKET 4	BUCKET 5
“tom”	“dan”	“leo”	“amy”	“zoe”	“mia”

Data Management

Hash table/map (6)

- Example
 - 3 buckets

KEY	HASH	HASH mod 3
"leo"	32	
"tom"	48	
"dan"	19	
"mia"	23	
"zoe"	46	
"amy"	39	

Data Management

Hash table/map (7)

- Example
 - 3 buckets

KEY	HASH	HASH mod 3
"leo"	32	2
"tom"	48	0
"dan"	19	1
"mia"	23	2
"zoe"	46	1
"amy"	39	0

BUCKET 0	BUCKET 1	BUCKET 2
"tom"	"dan"	"leo"
"amy"	"zoe"	"mia"

Data Management

Hash table/map (8)

- Cost to find “leo”?
 - 1
- Cost to find “amy”?
 - 2

KEY	HASH	HASH mod 3
“leo”	32	2
“tom”	48	0
“dan”	19	1
“mia”	23	2
“zoe”	46	1
“amy”	39	0

BUCKET 0	BUCKET 1	BUCKET 2
“tom”	“dan”	“leo”
“amy”	“zoe”	“mia”

Data Management

Hash table/map (9)

- Hash table/map
 - Hash size N
 - K number of buckets
 - $O(N/k)$

Data Management

Distributed Hash Table (1)

- Motivation
 - Load balancing
 - (Scale out)

Data Management

Distributed Hash Table (2)

- Example
 - $A \leftrightarrow \text{mod } 3 = 0$
 - $B \leftrightarrow \text{mod } 3 = 1$
 - $C \leftrightarrow \text{mod } 3 = 2$

KEY	HASH	HASH mod 3	A	B	C
"leo"	32	2			
"tom"	48	0			
"dan"	19	1			
"mia"	23	2			
"zoe"	46	1			
"amy"	39	0			

Data Management

Distributed Hash Table (3)

- Example
 - $A \leftrightarrow \text{mod } 3 = 0$
 - $B \leftrightarrow \text{mod } 3 = 1$
 - $C \leftrightarrow \text{mod } 3 = 2$

KEY	HASH	HASH mod 3	A	B	C
"leo"	32	2	"tom"	"dan"	"leo"
"tom"	48	0	"amy"	"zoe"	"mia"
"dan"	19	1			
"mia"	23	2			
"zoe"	46	1			
"amy"	39	0			

Data Management

Distributed Hash Table (4)

- Limitation
 - Rehashing
- Example
 - C leaves the system

KEY	HASH		A	B
"leo"	32			
"tom"	48			
"dan"	19			
"mia"	23			
"zoe"	46			
"amy"	39			

Data Management

Distributed Hash Table (5)

- Limitation
 - Rehashing
- Example
 - C leaves the system
 - $A \leftrightarrow \text{mod } 2 = 0$
 - $B \leftrightarrow \text{mod } 2 = 1$

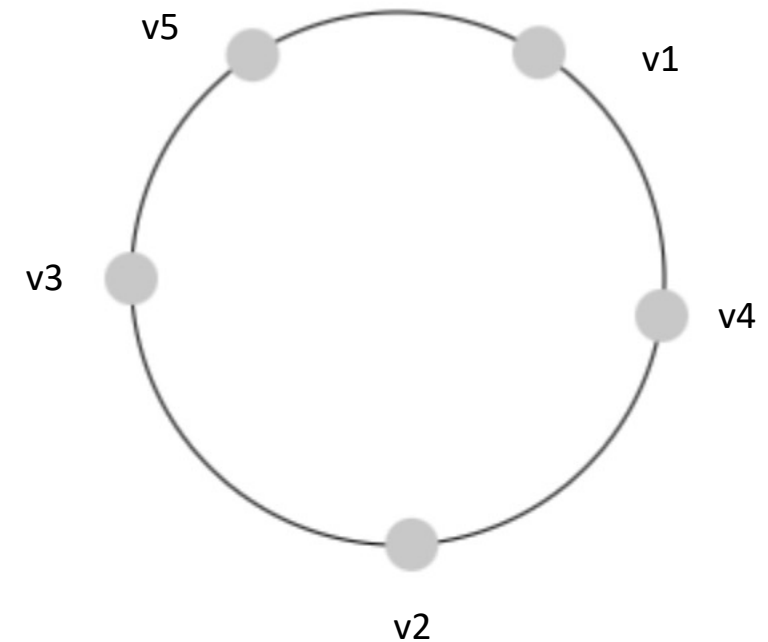
KEY	HASH	HASH mod 2	A	B
"leo"	32	0	"leo"	"dan"
"tom"	48	0	"tom"	"mia"
"dan"	19	1	"zoe"	"amy"
"mia"	23	1		
"zoe"	46	0		
"amy"	39	1		

Data Management

Consistent Hashing (1)

- Example

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123

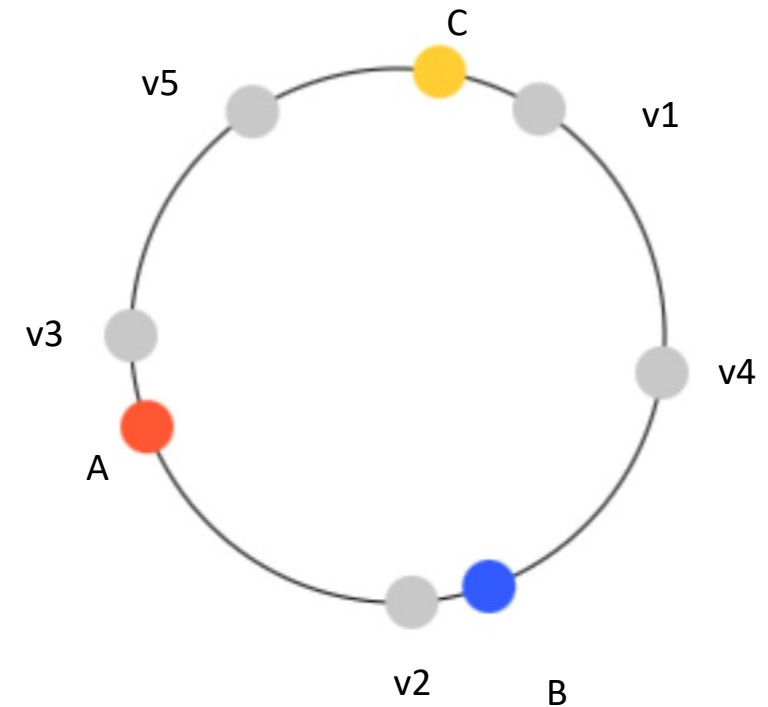


Data Management

Consistent Hashing (2)

- Example

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123
A	hA	200
B	hB	290
C	hC	81



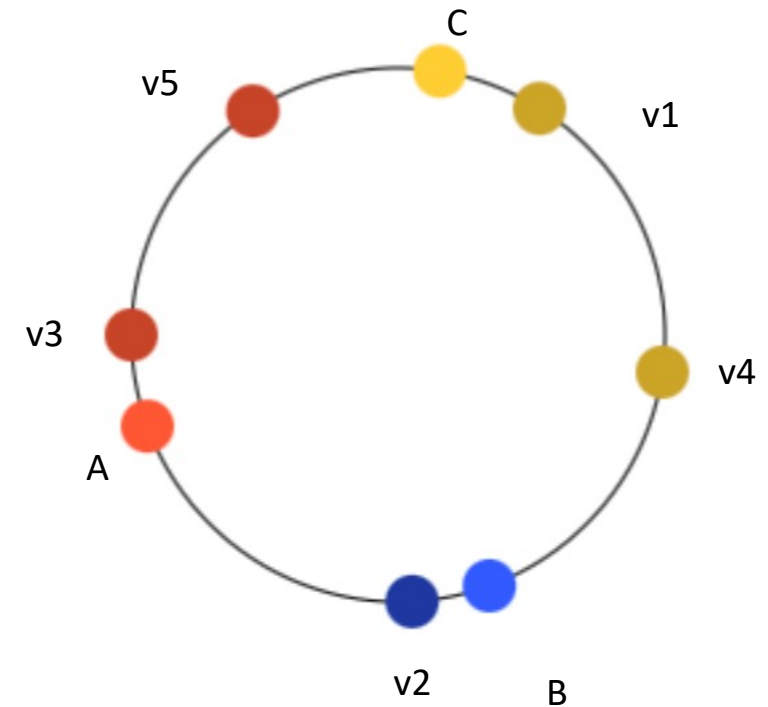
Data Management

Consistent Hashing (3)

- Example

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123
A	hA	200
B	hB	290
C	hC	81

NODE	KEYS
A	v3, v5
B	v2
C	v1, v4

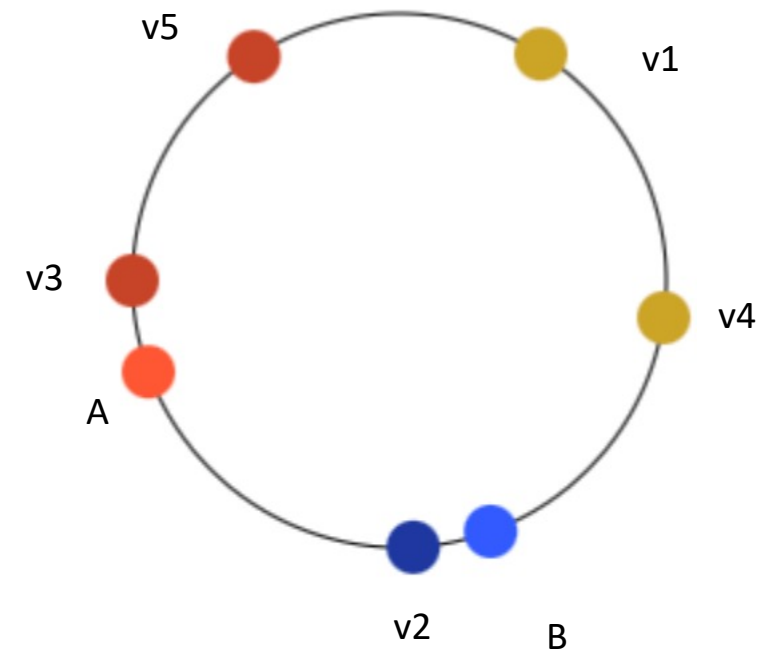


Data Management

Consistent Hashing (4)

- Example
 - Remove C

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123
A	hA	200
B	hB	290



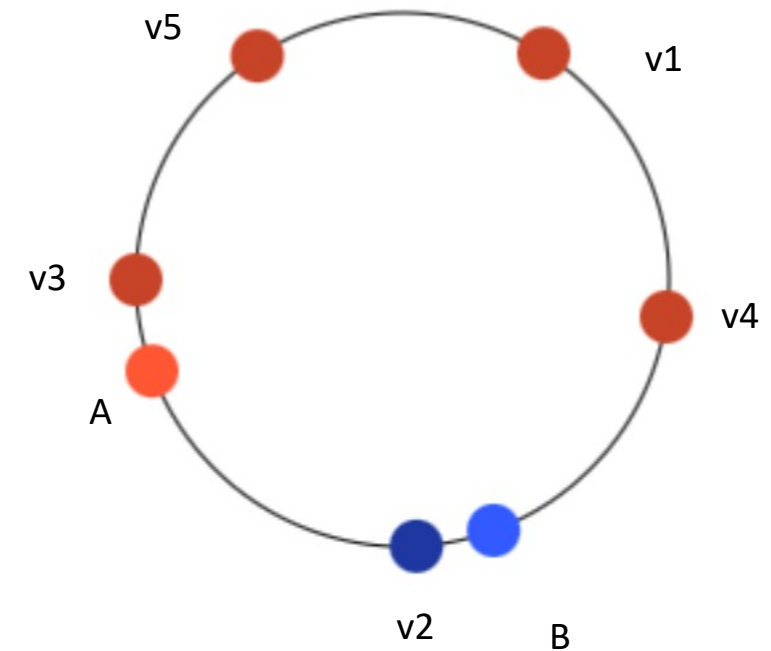
Data Management

Consistent Hashing (5)

- Example
 - Remove C

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123
A	hA	200
B	hB	290

NODE	KEYS
A	v3, v5, v1, v4
B	v2

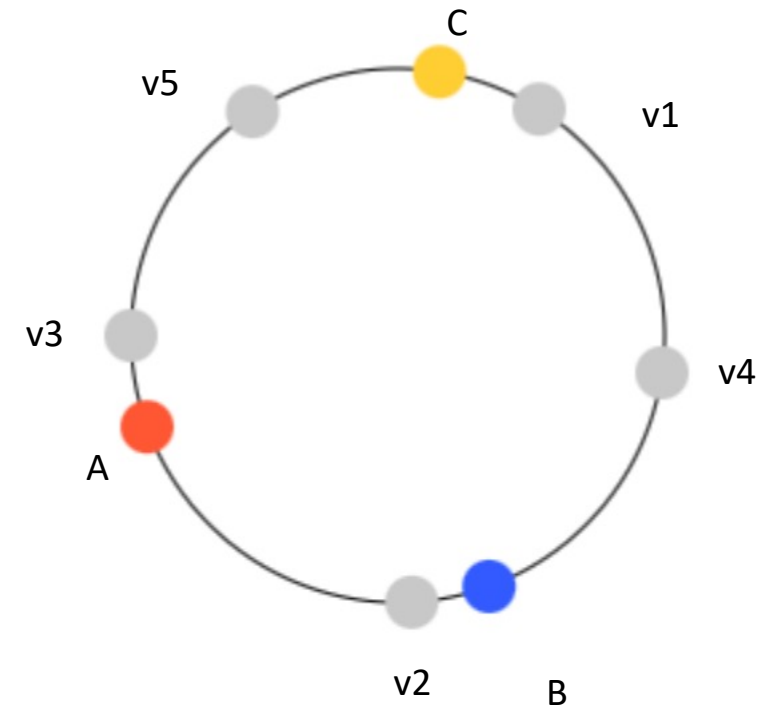


Data Management

Consistent Hashing (6)

- Example
 - Skewness

KEY	HASH	ANGLE
v1	h1	58
v2	h2	273
v3	h3	180
v4	h4	352
v5	h5	123
A	hA	200
B	hB	290
C	hC	81



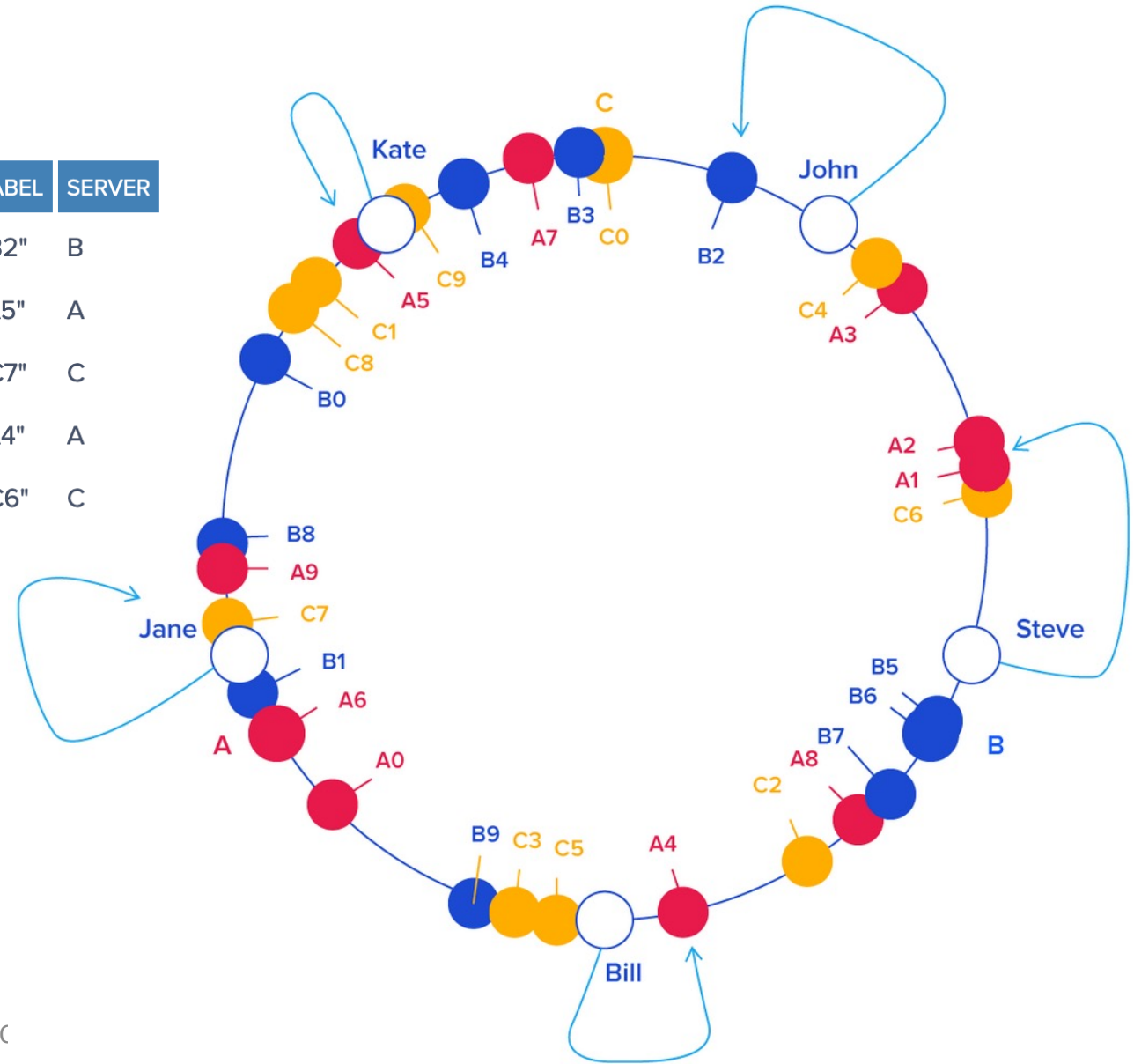
Data Management

Consistent Hashing (7)

- Example

KEY	HASH	ANGLE (DEG)
"C6"	408965526	14.7
"A1"	473914830	17
"A2"	548798874	19.7
"A3"	1466730567	52.8
"C4"	1493080938	53.7
"john"	1633428562	58.7
"B2"	1808009038	65
"C0"	1982701318	71.3
"B3"	2058758486	74.1
"A7"	2162578920	77.8
"B4"	2660265921	95.7
"C9"	3359725419	120.9
"kate"	3421657995	123.1
"A5"	3434972143	123.6

KEY	HASH	ANGLE (DEG)	LABEL	SERVER
"john"	1632929716	58.7	"B2"	B
"kate"	3421831276	123.1	"A5"	A
"jane"	5000648311	180	"C7"	C
"bill"	7594873884	273.4	"A4"	A
"steve"	9786437450	352.3	"C6"	C



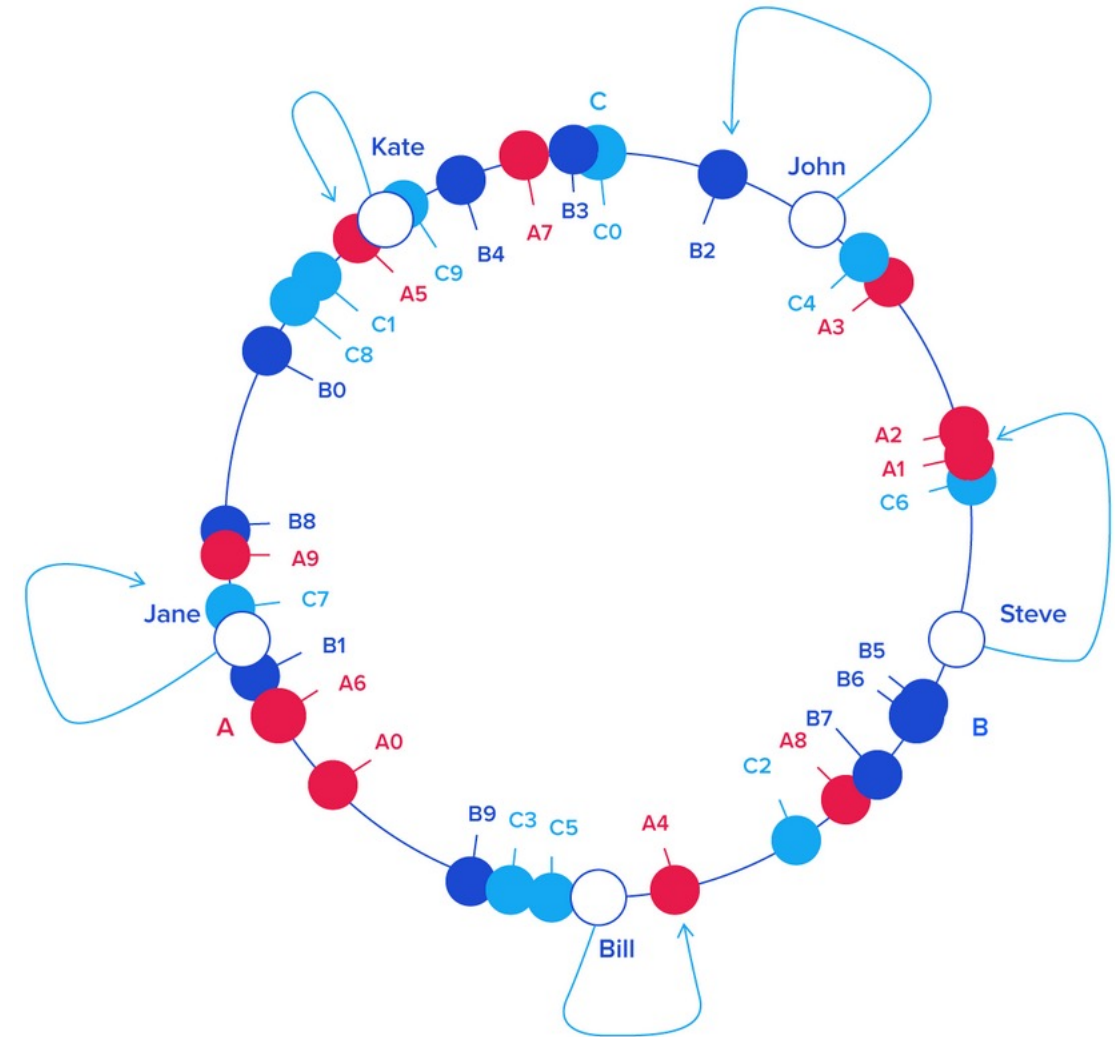
Data Management

Consistent Hashing (8)

- Example
 - Remove C

KEY	HASH	ANGLE (DEG)
"A1"	473914830	17
"A2"	548798874	19.7
"A3"	1466730567	52.8
"john"	1633428562	58.7
"B2"	1808009038	65
"B3"	2058758486	74.1
"A7"	2162578920	77.8
"B4"	2660265921	95.7
"kate"	3421657995	123.1
"A5"	3434972143	123.6
"B0"	4049028775	145.7
"B8"	4755525684	171.1
"A9"	4769549830	171.7
"jane"	5000799124	180
"B1"	5444659173	196

KEY	HASH	ANGLE (DEG)	LABEL	SERVER
"john"	1632929716	58.7	"B2"	B
"kate"	3421831276	123.1	"A5"	A
"jane"	5000648311	180	"B1"	B
"bill"	7594873884	273.4	"A4"	A
"steve"	9786437450	352.3	"A1"	A



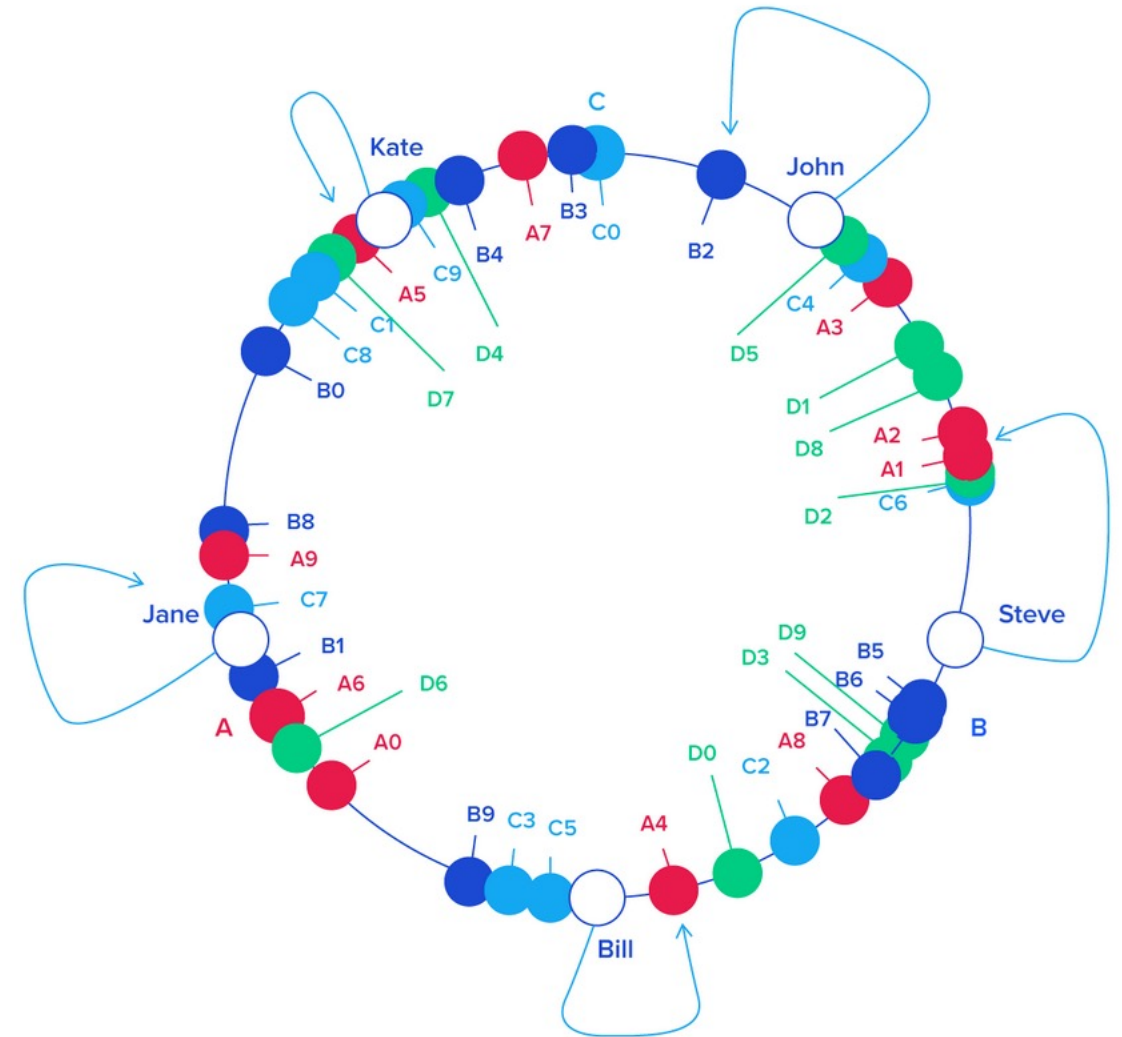
Data Management

Consistent Hashing (9)

- Example
 - Add D

KEY	HASH	ANGLE (DEG)
"D2"	439890723	15.8
"A1"	473914830	17
"A2"	548798874	19.7
"D8"	796709216	28.6
"D1"	1008580939	36.3
"A3"	1466730567	52.8
"D5"	1587548309	57.1
"john"	1633428562	58.7
"B2"	1808009038	65
"B3"	2058758486	74.1
"A7"	2162578920	77.8
"B4"	2660265921	95.7
"D4"	2909395217	104.7
"kate"	3421657995	123.1
"A5"	3434972143	123.6

KEY	HASH	ANGLE (DEG)	LABEL	SERVER
"john"	1632929716	58.7	"B2"	B
"kate"	3421831276	123.1	"A5"	A
"jane"	5000648311	180	"B1"	B
"bill"	7594873884	273.4	"A4"	A
"steve"	9786437450	352.3	"D2"	D



Bibliography

- P. Rigaux, class on Cassandra Data Model, CNAM
- P. Nerzic, class on Cassandra, IUT de Lannion
- <http://cassandra.apache.org/doc/latest/>
- [LM10] Lakshman, A., & Malik, P. (2010). Cassandra: a decentralized structured storage system. Operating Systems Review, 44(2), 35–40.
- [KLL97] Karger, D. et al. (1997). Consistent Hashing and Random Trees: Distributed Caching Protocols for Relieving Hot Spots on the World Wide Web. Proceedings of the Symposium on the Theory of Computing (STOC) (pp. 654–663). El Paso, Texas, USA.
- <https://www.toptal.com/big-data/consistent-hashing>

Questions?